

Software Design Description

Aerovision : Drone Detection with Machine Learning



Submitted by

Rimsha Malik (S23BINFT1M01062)

Submitted to

Dr. Syed Ali Nawaz Shah

**Department of Information Technology
Faculty of Computing
The Islamia University of Bahawalpur**

Summary

This Software Design Document (SDD) describes the internal architecture and detailed design of AeroVision, a machine learning-based real-time drone detection system. The document covers the overall application architecture explaining how the Streamlit interface, the detection module, and the YOLOv8 model fit together. The Data Model section describes how data is managed in memory during a session using Python dictionaries and Streamlit session state, since AeroVision uses no relational database. The User Interface section documents the design philosophy, screen layout, and controls for all five pages of the application. All required diagrams including the System Context Diagram, Data Flow Diagrams, Activity Diagram, Sequence Diagram, Component Diagram, and Class Diagram are included in the relevant sections. This document is intended to guide the development of AeroVision and serve as a design reference for the project supervisor and developers.

Table of Content

Summary.....	I
Table of Content.....	II
1. Application Architecture.....	1
1.1 Architecture Pattern.....	1
1.2 Components and Their Roles.....	1
1.3 Key Functions in detect.py.....	2
1.4 External Libraries and APIs.....	2
1.5 Coding Conventions.....	3
1.6 UML Diagrams.....	3
1.6.1 Figure 1 — System Context Diagram.....	3
1.6.2 Figure 2 — Data Flow Diagram.....	4
1.6.3 Figure 3 — Sequence Diagram.....	5
1.6.4 Figure 4 — Activity Diagram.....	6
1.6.5 Figure 5 — Component Diagram.....	7
1.6.6 Figure 6 — Class Diagram.....	8
1.6.7 Figure 7 — System Use Case Diagram.....	9
2. Data Model Schema.....	10
2.1 Primary Data Object — DetectionResult.....	10
2.2 Session History.....	11
2.3 Session State Variables.....	11
2.4 Note on Database Absence.....	11
3. User Interface.....	12
3.1 Design and Interface Specifications.....	12
3.1.1 Color Palette and Visual Hierarchy.....	12
3.1.2 Typography.....	13
3.1.3 Layout and Structural Framework.....	13
3.2 List of Pages and Screen Descriptions.....	14
Page 1 — Home.....	14
Page 2 — Upload and Detect.....	14
Page 3 — Results.....	14
Page 4 — History.....	14
Page 5 — About.....	14
3.3 Controls, Input Constraints, and Feedback.....	15
3.4 Screen Images.....	16
Appendix A: Glossary.....	21
Appendix B: Design Models.....	22
References.....	23

1. Application Architecture

AeroVision follows a two-layer architecture that cleanly separates the user interface from the detection logic. The Presentation Layer is handled entirely by Streamlit and the Detection Layer is encapsulated in a standalone Python module called `detect.py`. The two layers communicate through direct Python function calls with no HTTP API, no web server-to-server communication, and no database involved at runtime.

1.1 Architecture Pattern

The system follows a simplified MVC pattern. Streamlit acts as both the View and the Controller. `detect.py` acts as the Model. This separation means that the entire detection algorithm can be replaced or retrained without touching any of the interface code, and the interface can be redesigned without touching any of the ML code.

1.2 Components and Their Roles

`streamlit_app.py`: Main entry point. Manages sidebar navigation and routes the user to the correct page module. Initialises `st.session_state` variables on first run.

`pages/home.py`: Renders the Home page with the welcome hero card, three feature cards, how-it-works steps, and system information strip. Fully static — no user input.

`pages/upload.py`: Handles the file upload flow. Validates the uploaded file for type and size. Provides the confidence threshold slider and show-labels toggle. Triggers the detection pipeline when the Run Detection button is clicked.

`pages/results.py`: Reads the detection result from `st.session_state` and renders the annotated output image, detection metrics, detections table, confidence breakdown bars, download button, and Detect Another button.

`pages/about.py`: Static page showing project details, technology stack, model performance metrics, and system flow summary.

`detect.py`: Core detection module. Contains all ML and image processing logic. Called by `upload.py` when the user triggers detection.

1.3 Key Functions in detect.py

load_model(model_path): Loads the YOLOv8 model from the given path using the Ultralytics YOLO class. Decorated with `st.cache_resource` so the model is loaded exactly once at application startup and reused on every subsequent request without reading from disk again. Raises `FileNotFoundError` if `best.pt` is not found.

run_detection(image, model, threshold): Accepts a PIL Image object and the loaded model. Converts the image to a NumPy array and passes it to the model with the confidence threshold. Extracts bounding box coordinates (x1, y1, x2, y2), class labels, and confidence scores from the YOLO results object. Calls `draw_boxes()` to annotate the image. Returns a `DetectionResult` dictionary.

draw_boxes(image_array, detections, show_labels): Iterates over each detection in the list. Uses `cv2.rectangle` to draw a bounding box around each detected drone. If `show_labels` is `True`, uses `cv2.putText` to overlay the confidence score as a label on each box. Returns the annotated image as a NumPy array which is then converted back to PIL for display.

1.4 External Libraries and APIs

Ultralytics YOLOv8: Provides the YOLO class which handles the entire model loading and inference pipeline internally. Calling `model(image, conf=threshold)` runs preprocessing, inference, and postprocessing in one step and returns a `Results` object containing detected boxes and scores.

OpenCV (cv2): Used to draw bounding boxes and text labels on images after detection. `cv2.rectangle` draws each box and `cv2.putText` prints the confidence score. Also used for color space conversion between BGR and RGB.

Streamlit: Renders the entire web interface. Key functions used: `st.file_uploader` for image uploads, `st.image` for displaying images, `st.metric` for stat cards, `st.columns` for multi-panel layouts, `st.download_button` for saving results, `st.slider` and `st.toggle` for settings, `st.session_state` for sharing data across pages, and `st.cache_resource` for model caching.

Pillow (PIL): Used to open uploaded image files into PIL Image objects and to convert between PIL Image format and NumPy arrays for OpenCV compatibility.

NumPy: Handles image array manipulation during the conversion pipeline between PIL, OpenCV, and YOLO input formats.

1.5 Coding Conventions

- All Python files follow PEP 8 naming conventions throughout
- Functions use lowercase_with_underscores naming style
- Constants such as model path and default threshold are defined at the top of each file
- Each page module contains a single render() function that is called by streamlit_app.py
- Type hints are used for all function parameters and return values
- Comments are written in plain English to explain non-obvious logic

1.6 UML Diagrams

1.6.1 Figure 1 — System Context Diagram

The System Context Diagram shows AeroVision as the central system bounded by a rectangle, with all external entities shown outside the boundary. The General User interacts directly with the system by uploading images and receiving detection results. The YOLOv8 Model (best.pt) is loaded at startup. The Kaggle Dataset was used for offline training only and is not accessed at runtime. The Python/Streamlit Runtime hosts the entire application.

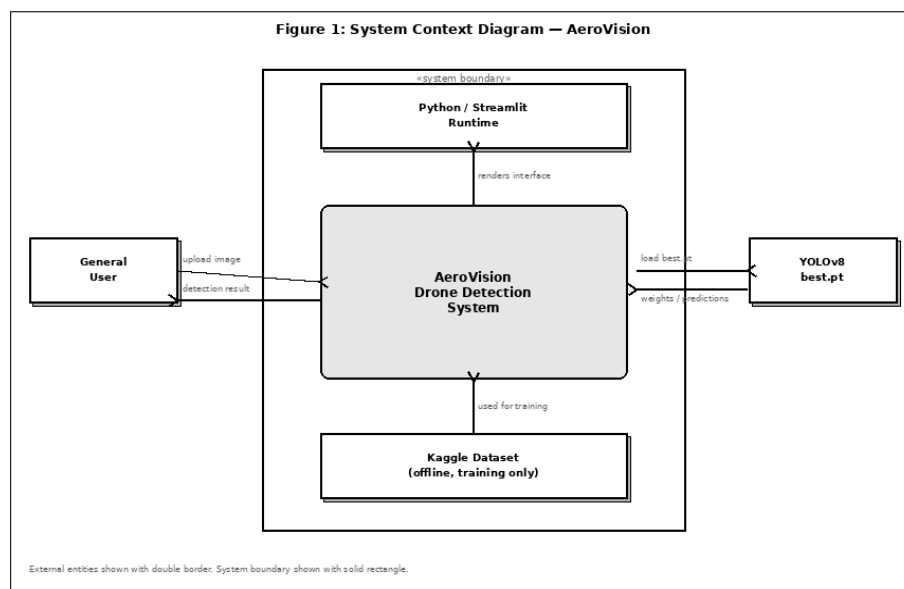


Figure 1: System Context Diagram — AeroVision

1.6.2 Figure 2 — Data Flow Diagram

The DFD is shown at two levels. Level 0 shows the entire AeroVision system as a single process receiving image input from the user and returning detection results. Level 1 decomposes this into four sub-processes: 1.1 Validate File checks the format and size; 1.2 Run Detection passes the image to YOLOv8; 1.3 Annotate Image uses OpenCV to draw bounding boxes; 1.4 Display Results renders the output. Data stores show where session state data is held.

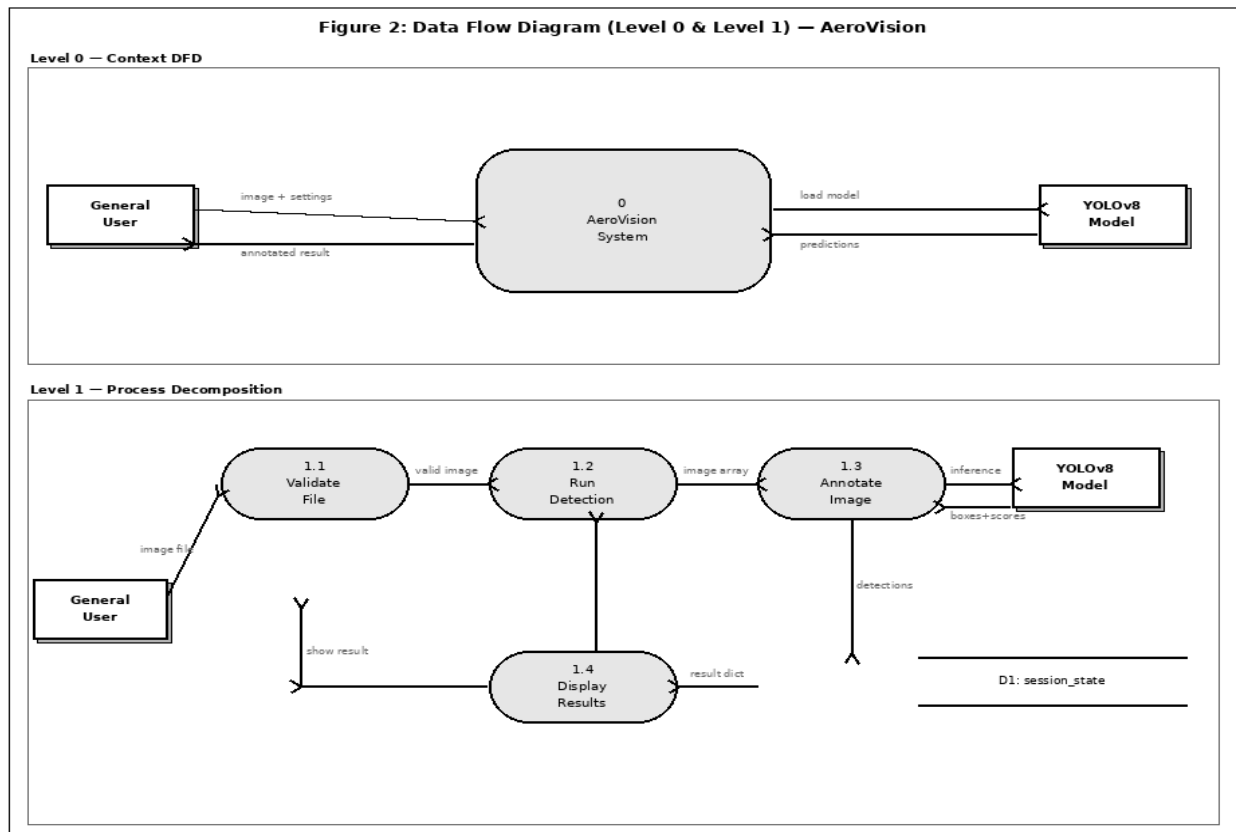


Figure 2: Data Flow Diagram (Level 0 and Level 1) — AeroVision

1.6.3 Figure 3 — Sequence Diagram

The Sequence Diagram shows the message flow between the four participants: General User, Streamlit UI, detect.py, and YOLOv8 Model. Solid arrows represent synchronous calls and dashed arrows represent return messages. An alt block covers two scenarios: when drones are found the annotated image is displayed; when no drones are found the no-drone message is shown.

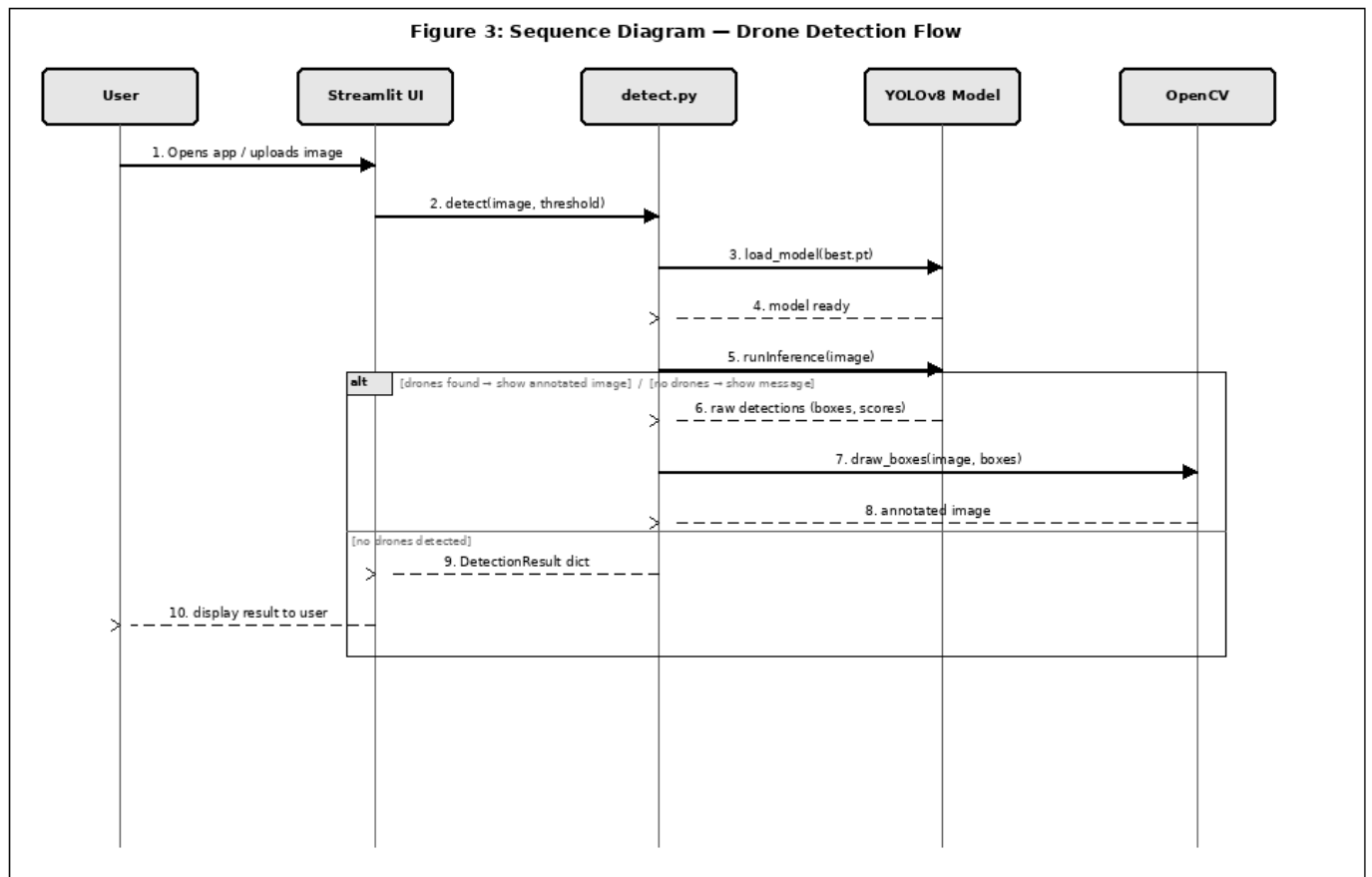


Figure 3: Sequence Diagram — Drone Detection Flow

1.6.4 Figure 4 — Activity Diagram

The Activity Diagram shows the complete control flow of AeroVision from app startup through image upload, file validation, model inference, result display, and session end. Two diamond decision nodes show the branching: first for file validation (valid or retry), second for detection outcome (drones found or no drones message). A loop allows the user to detect another image without restarting the app.

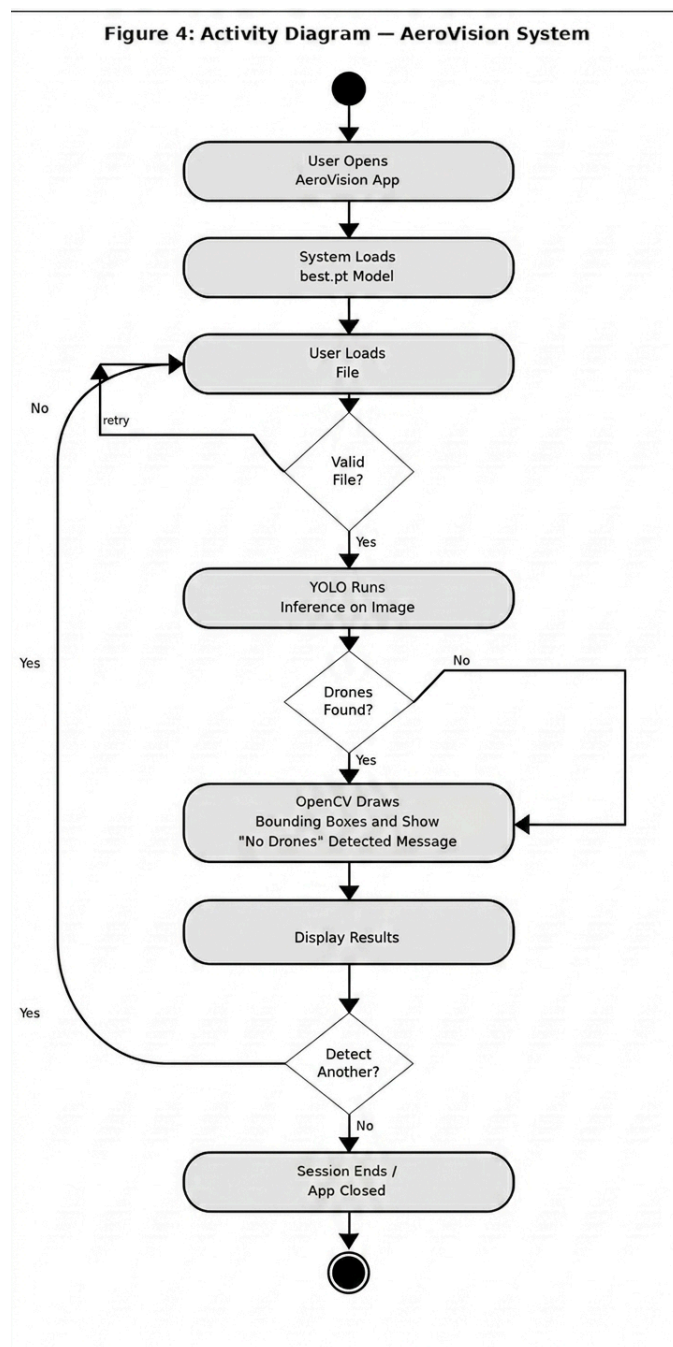


Figure 4: Activity Diagram — Full System Control Flow

1.6.5 Figure 5 — Component Diagram

The Component Diagram shows the three subsystems of AeroVision: the Streamlit Presentation Layer containing all page modules, the Detection Layer containing detect.py and OpenCV, and the External layer containing YOLOv8 best.pt, Pillow, and NumPy. Each component is drawn with the UML component icon. Arrows show the direction of calls and data flow between subsystems.

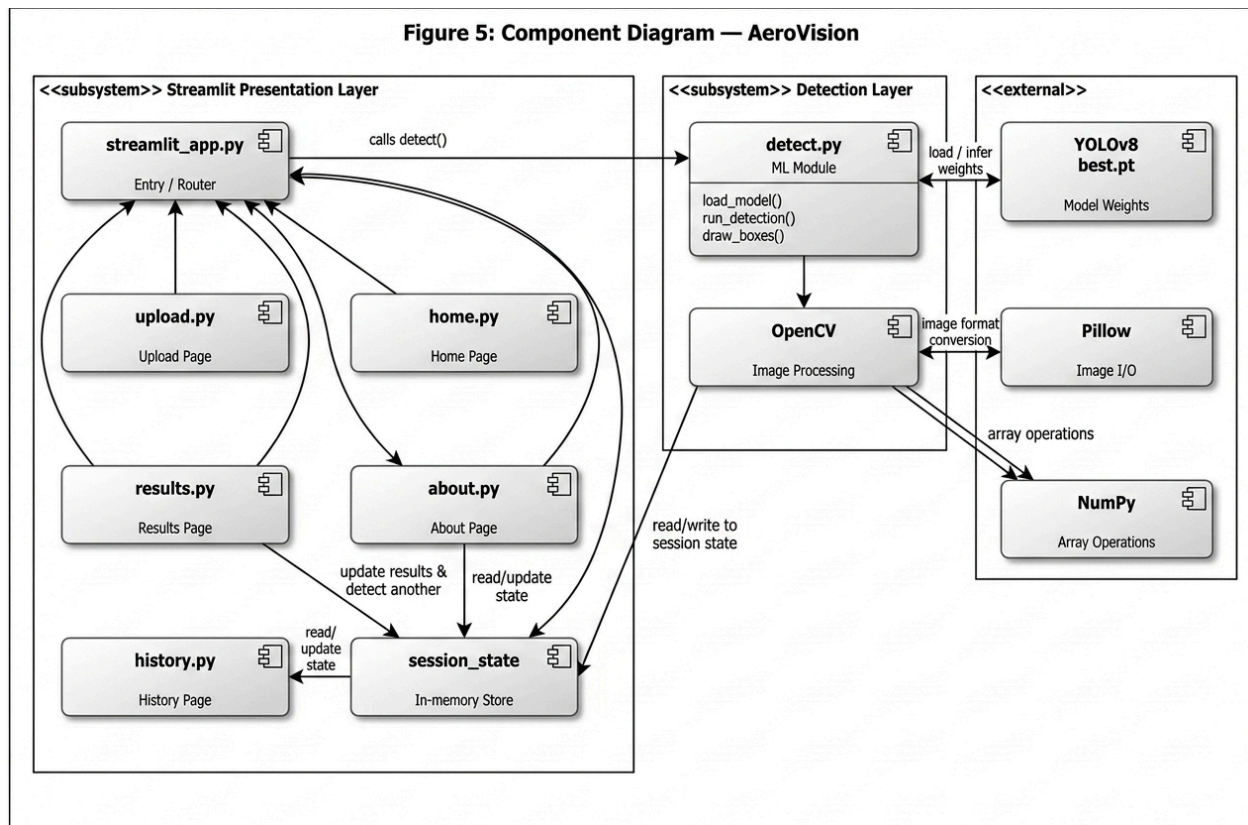


Figure 5: Component Diagram — AeroVision Module Structure

1.6.6 Figure 6 — Class Diagram

The Class Diagram shows the five main classes: AeroVisionApp, DetectionEngine, DetectionResult, ResultsPage, HistoryPage, and UploadPage, plus the shared SessionState. Each class shows its attributes and methods with UML visibility indicators. Relationships shown include: uses (AeroVisionApp to DetectionEngine), creates (DetectionEngine creates DetectionResult), reads (ResultsPage reads DetectionResult), and reads/writes (all pages through SessionState).

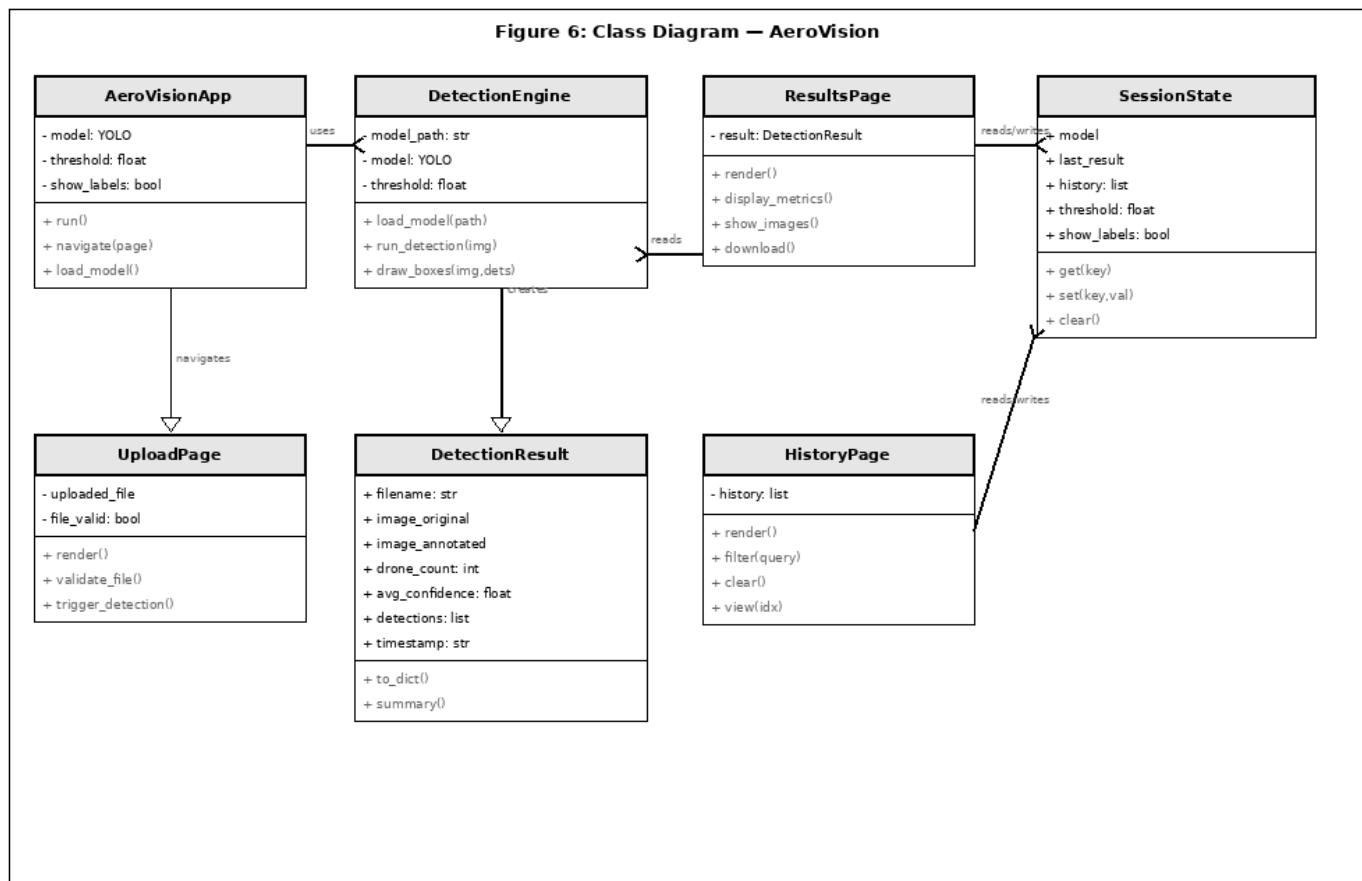


Figure 6: Class Diagram — AeroVision Classes and Relationships

1.6.7 Figure 7 — System Use Case Diagram

The use cases below describe the key interactions between the user and AeroVision. The primary actor in all cases is the General User, who accesses the system through the Streamlit web interface running locally on their machine. The system itself acts as a secondary actor by loading the model and processing the image.

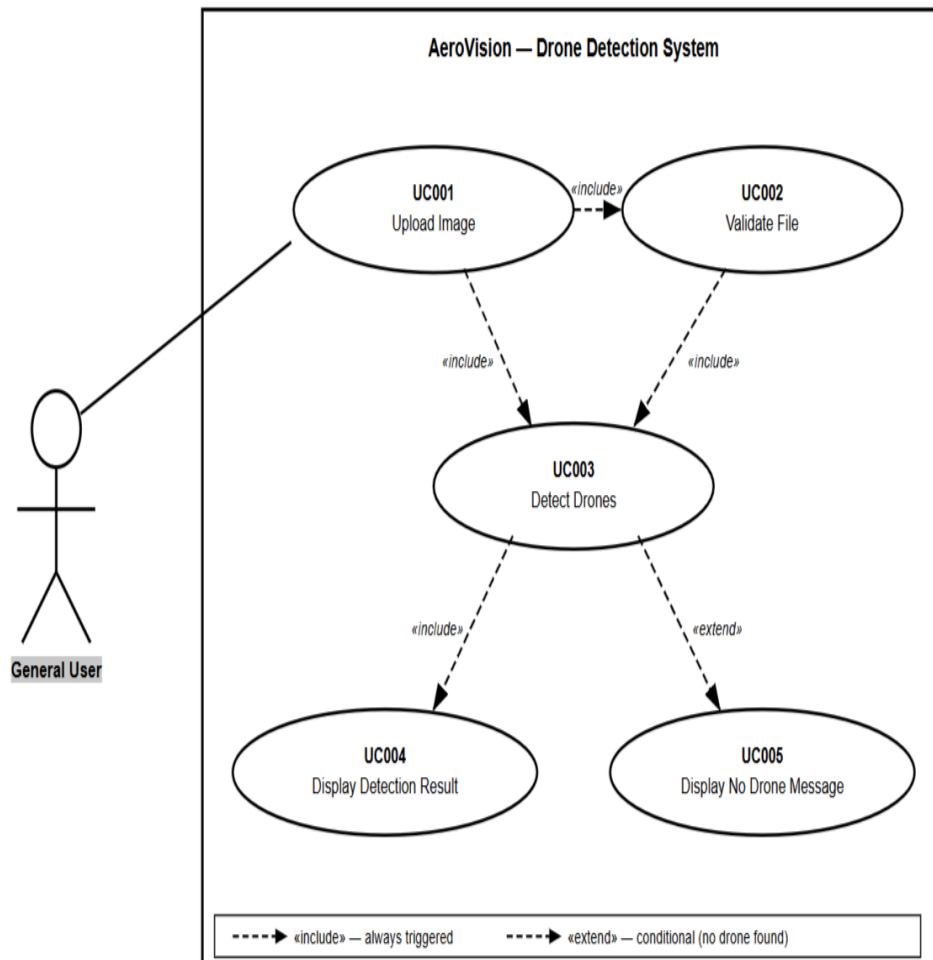


Figure 6: System Use Case Diagram — Interaction b/w User & AeroVision

2. Data Model Schema

AeroVision does not use a relational database or any persistent storage. All data is managed in memory during the session using Python data structures and Streamlit session state. When the application is closed or the browser is refreshed, all session data is cleared automatically. The following describes the complete in-memory data model used across the application.

2.1 Primary Data Object — `DetectionResult`

Each time the user runs detection, the system creates a `DetectionResult` object stored as a Python dictionary. This object is the central data carrier passed between `detect.py` and the Streamlit pages. It contains the following fields:

1. **filename (string):** The name of the uploaded image file, e.g. `drone_sample.jpg`
2. **image_original (Image):** The original uploaded image before any annotation is applied
3. **image_annotated (Image):** The output image with bounding boxes drawn by OpenCV around each detected drone
4. **drone_count (integer):** Total number of drones detected in the image by the model
5. **avg_confidence (float):** The average confidence score across all detections expressed as a decimal between 0 and 1
6. **detections (list of dictionaries):** Each item in the list represents one detected drone and contains: `id` (integer, detection index starting from 1), `label` (string, always the value `drone`), `confidence` (float between 0 and 1), and `box` (tuple of four integers `x1`, `y1`, `x2`, `y2` representing the top-left and bottom-right corners of the bounding box in pixel coordinates)
7. **timestamp (string):** The date and time when the detection was run, captured using Python `datetime.now().strftime()`

2.2 Session History

The application maintains a list stored in `st.session_state[history]` that accumulates `DetectionResult` dictionaries throughout the current session. Each time a detection completes successfully, the new result dictionary is appended to this list. The History page reads directly from this list to populate the session history table. The list is initialised as an empty list when the app first loads and is cleared entirely when the app is closed or the browser session ends.

2.3 Session State Variables

The following variables are stored in `st.session_state` and are shared across all pages of the application:

model: The loaded YOLO model object returned by `load_model()`. Cached using `st.cache_resource` so it persists across all page navigations without being reloaded from disk on each request.

last_result: The most recent `DetectionResult` dictionary. Written by `upload.py` after detection completes and read by `results.py` to render the output.

history: Python list of all `DetectionResult` dictionaries produced during the current session.

threshold: Float. The confidence threshold set by the user on the Upload page slider. Default value is 0.50. Passeded to `run_detection()` on each inference call.

show_labels: Boolean. Whether to display confidence score labels on top of bounding boxes in the annotated output image. Default is True.

2.4 Note on Database Absence

Since AeroVision uses no relational database, there are no entity-relationship diagrams, no SQL table schemas, and no data dictionary entries with field types and constraints. The Python dictionary structures and session state variables described above constitute the complete data specification for this system. If a persistent history feature is added in future versions, a lightweight database such as SQLite would be the recommended addition.

3. User Interface

AeroVision provides a clean and professional web interface built entirely using Streamlit. The interface is designed to be immediately usable by a non-technical user within 30 seconds of opening the application, without requiring any instructions. A dark sidebar provides consistent navigation across all five pages and all primary content appears in white card panels on a light grey background.

3.1 Design and Interface Specifications

3.1.1 Color Palette and Visual Hierarchy

The application employs a professional, high-contrast visual identity designed for clarity and focus. The palette is strictly managed to signify functional states:

- **Sidebar:** Deep Navy (#1E1E2E) provides a grounded, static navigation anchor.
- **Background:** Light Grey (#F5F6FA) for the main content area, providing optimal readability against white containers.
- **Primary Actions:** Red (#DC2626) is reserved for high-intent triggers (e.g., "Run Detection" button) and primary bounding box annotations.
- **Secondary Actions:** Blue (#2563EB) denotes secondary navigation, process initiation, and download actions.
- **Status & Feedback:** Green (#16A34A) is utilized for "Model Ready" status indicators, "Complete" status markers, and "No Drone" classifications. Orange (#EA580C) is applied to medium-confidence detection labels and metrics to differentiate them from the primary red high-confidence indicators.

3.1.2 Typography

The application maintains a clean, minimalist aesthetic by relying on the system's native **Roboto** font stack.

- **Headers:** `st.title()` and `st.subheader()` are utilized for consistent H1 and H2 styling.
- **Body & Metadata:** `st.write()` and `st.caption()` are used for body text and supplementary metadata, respectively.
- **Constraint:** No external custom fonts are imported to ensure rapid load times and consistent rendering across environments.

3.1.3 Layout and Structural Framework

The application is built for desktop-first data visualization, requiring a minimum screen width of **1280px**.

- **Page Configuration:** `st.set_page_config(layout="wide")` is strictly enabled across all pages to maximize workspace for image comparison and data tables.
- **Responsive Strategy:** The interface is **not optimized for mobile devices**. No responsive breakpoints are defined; users accessing the system on displays narrower than 1280px are expected to utilize horizontal scrolling.
- **Grid System:** Multi-panel layouts are achieved using `st.columns()`.
 - **Home Page:** Utilizes a three-column grid for feature summary cards.
 - **Results/Upload Pages:** Utilizes a dual-column layout to facilitate side-by-side comparison between input sources and processed output visualizations.

3.2 List of Pages and Screen Descriptions

Page 1 — Home

The landing page of AeroVision. Gives the user an overview of the system and a direct path to start detection. Contains: a full-width dark hero card with the application name, tagline, and a Start Detection button; three feature cards in a row showing Upload Image, Auto Detection, and Instant Results with icons and descriptions; a How It Works section with four numbered steps shown in columns; and a system information strip at the bottom listing the framework, interface, dataset, and output type.

Page 2 — Upload and Detect

The primary interactive page. Left panel contains the file upload area, file info display after selection, detection settings, and the Run Detection button. Right panel shows a live preview of the uploaded image before detection. Controls: `st.file_uploader` accepts JPG, JPEG, PNG files. `st.slider` for the confidence threshold (0.10 to 1.00, default 0.50). `st.toggle` for the Show Labels option. `st.button` for Run Detection. A progress spinner (`st.spinner`) appears during inference.

Page 3 — Results

Displays the output of the most recent detection run. Contains: four summary metric cards using `st.metric` showing Drones Detected, Average Confidence, Image Size, and Status; two images side by side using `st.columns` — the original and the annotated output with bounding boxes; a `st.dataframe` table showing each detection with ID, Label, Confidence, and Bounding Box coordinates; a confidence breakdown bar chart; a Download Result button using `st.download_button`; and a Detect Another button.

Page 4 — History

Displays all detection results from the current session only. A yellow info banner at the top reminds the user that records clear when the app is closed. Contains: a `st.text_input` search box that filters records by filename in real time; filter buttons for All, Drone Found, and No Drone; a Clear History button; and a table of all session records showing filename, timestamp, result, confidence, and a View button to reload any past result into the Results page.

Page 5 — About

A fully static informational page. Contains a dark hero card with the app name and version badges, three columns showing Project Details, Technology Stack, and Model Performance metrics, a system flow strip showing the four workflow steps, and a footer with project copyright and a GitHub link.

3.3 Controls, Input Constraints, and Feedback

st.file_uploader: Accepts JPG, JPEG, and PNG file types only. A manual size check in Python rejects files over 10MB before passing to the model. On valid selection, shows st.success with filename and size. On invalid file type, shows st.error. On an oversized file, shows st.error. The Run Detection button remains disabled until a valid file is loaded.

Confidence Threshold Slider: Range: 0.10 to 1.00. Default: 0.50. Step: 0.05. Value is stored in st.session_state[threshold] and passed to run_detection() on each call. Accepts only values within the defined range — no free text input.

Show Labels Toggle: Boolean on/off. Default: True. When off, bounding boxes are drawn without confidence score text labels. Value stored in st.session_state[show_labels].

Run Detection Button: Disabled state until a valid image has been uploaded and passed validation. On click: shows st.spinner during inference, calls run_detection(), stores result in st.session_state[last_result], appends to st.session_state[history], then navigates to the Results page using st.switch_page.

Download Button: Available on the Results page. Converts the annotated PIL Image to JPEG bytes using io.BytesIO and passes to st.download_button. File is saved to the user's device with the original filename prefixed with annotated_.

Search Box on History Page: st.text_input with no minimum length. Filters the history list in real time using Python list comprehension matching the search string against the filename field of each DetectionResult.

Clear History Button: Resets st.session_state[history] to an empty list and calls st.rerun() to refresh the page. Shows st.success confirmation message.

Model Not Found Error: If best.pt is not found in the model/ directory when the app starts, the application shows st.error on all pages and disables the Run Detection button until the file is present.

3.4 Screen Images

3.4.1 Home Screen

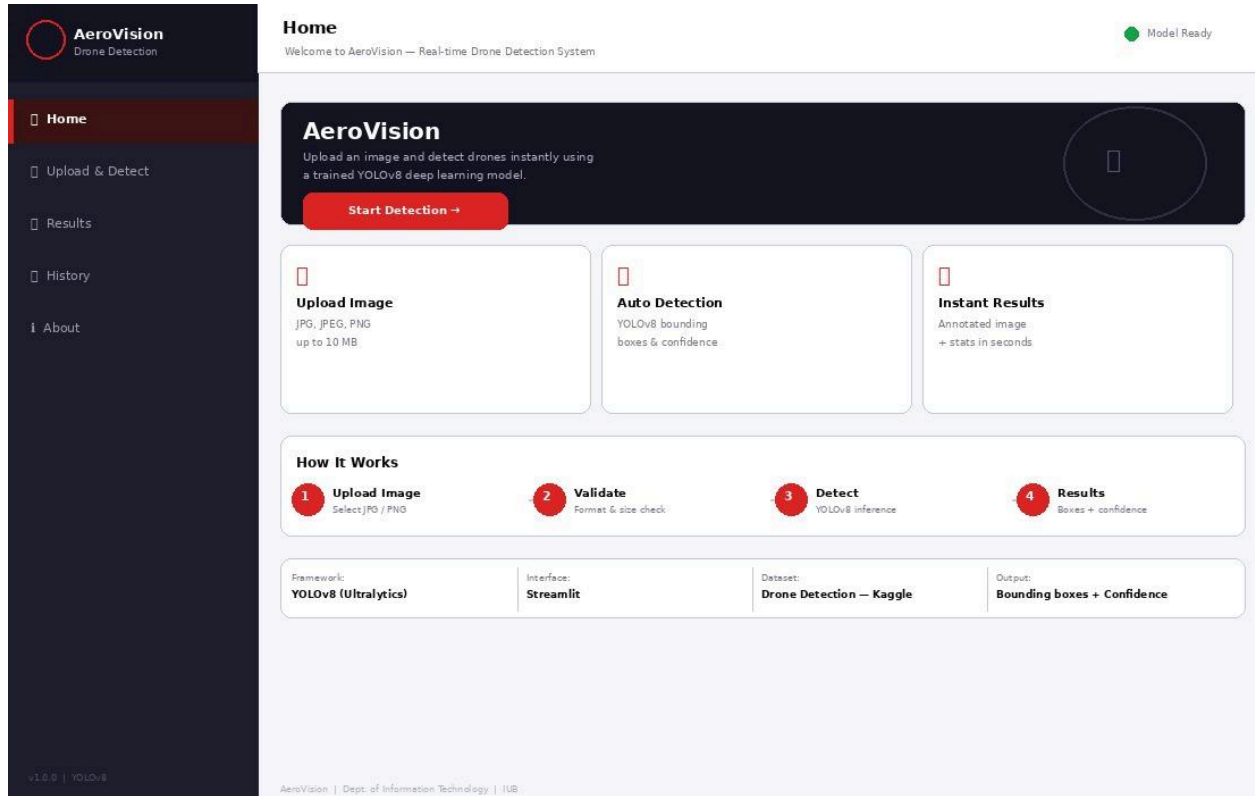


Figure 7 : Home Screen UI

3.4.2 Upload and Detect Screen

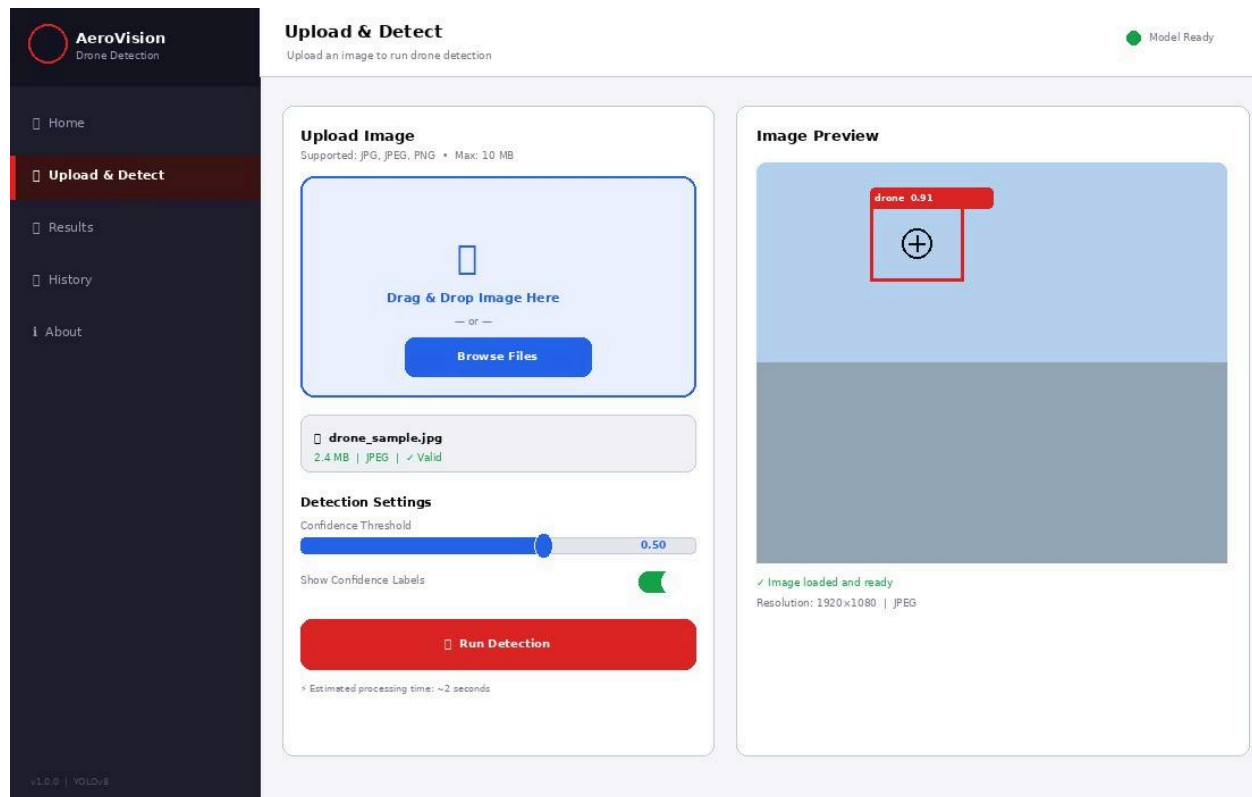


Figure 8 : Upload and Detect Screen UI

3.4.3 Results Screen

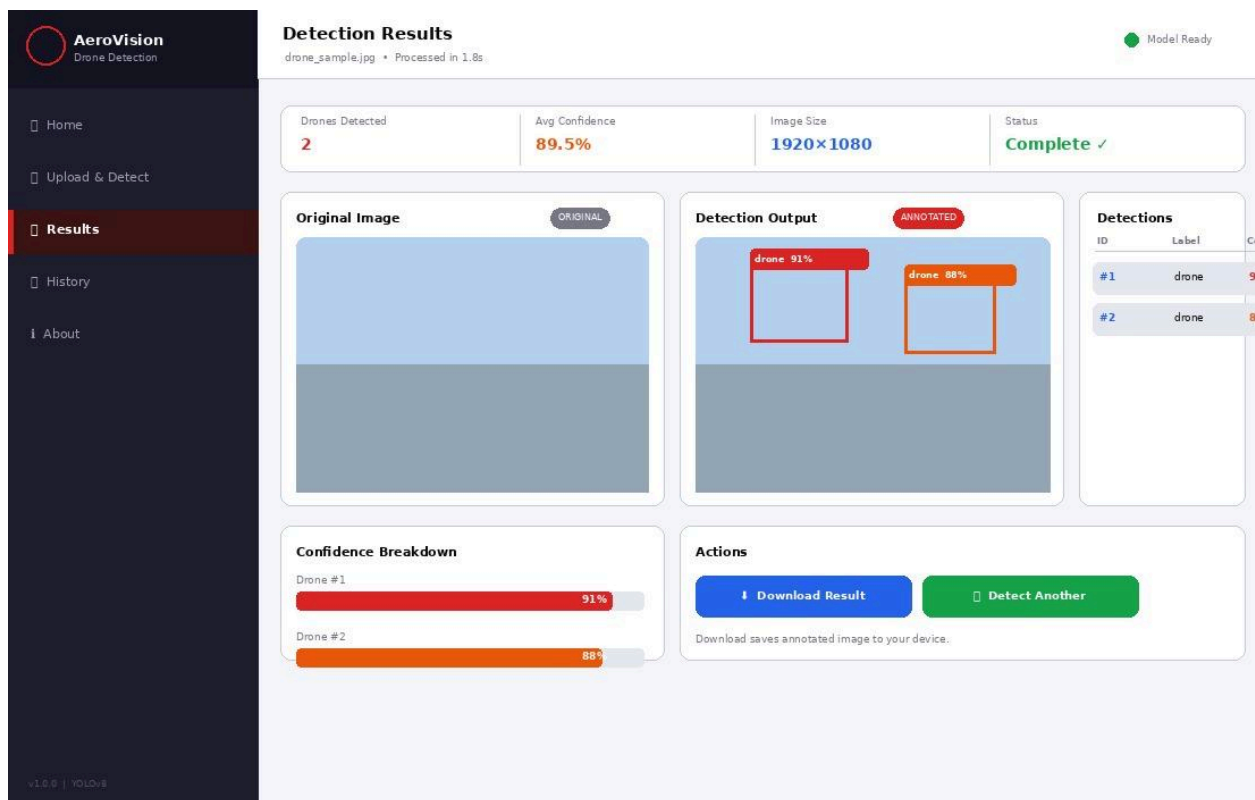


Figure 9 : Results Screen UI

3.4.4 History Screen

AeroVision
Drone Detection

About AeroVision
Project information and system details

Model Ready

AeroVision
Drone Detection with Machine Learning
v1.0.0 YOLOv8 Streamlit

Project Details

Title:	AeroVision: Drone Detection
Student:	Rimsha Malik
Roll No:	F22BINF1M01062
Supervisor:	Dr. Syed Ali Nawaz Shah
Dept:	Information Technology
Uni:	IUB Bahawalpur

Technology Stack

Language:	Python 3.10
Detection:	YOLOv8 (Ultralytics)
Interface:	Streamlit
CV Lib:	OpenCV
Training:	Kaggle Notebooks
Dataset:	Drone Detection (Kaggle)

Model Performance

Architecture:	YOLOv8n
Classes:	1 (drone)
mAP@0.5:	87.3 %
Precision:	91.2 %
Recall:	84.6 %
Weights:	best.pt

System Flow

- 1 Upload Image
Select JPG / PNG
- 2 Validate File
Format & size check
- 3 Detect Drones
YOLOv8 inference
- 4 Show Results
Boxes + confidence

© 2025 AeroVision | Final Year Project | Islamia University of Bahawalpur

[View on GitHub](#)

v1.0.0 | YOLOv8

Figure 10 : History Screen UI

3.4.5 About Screen

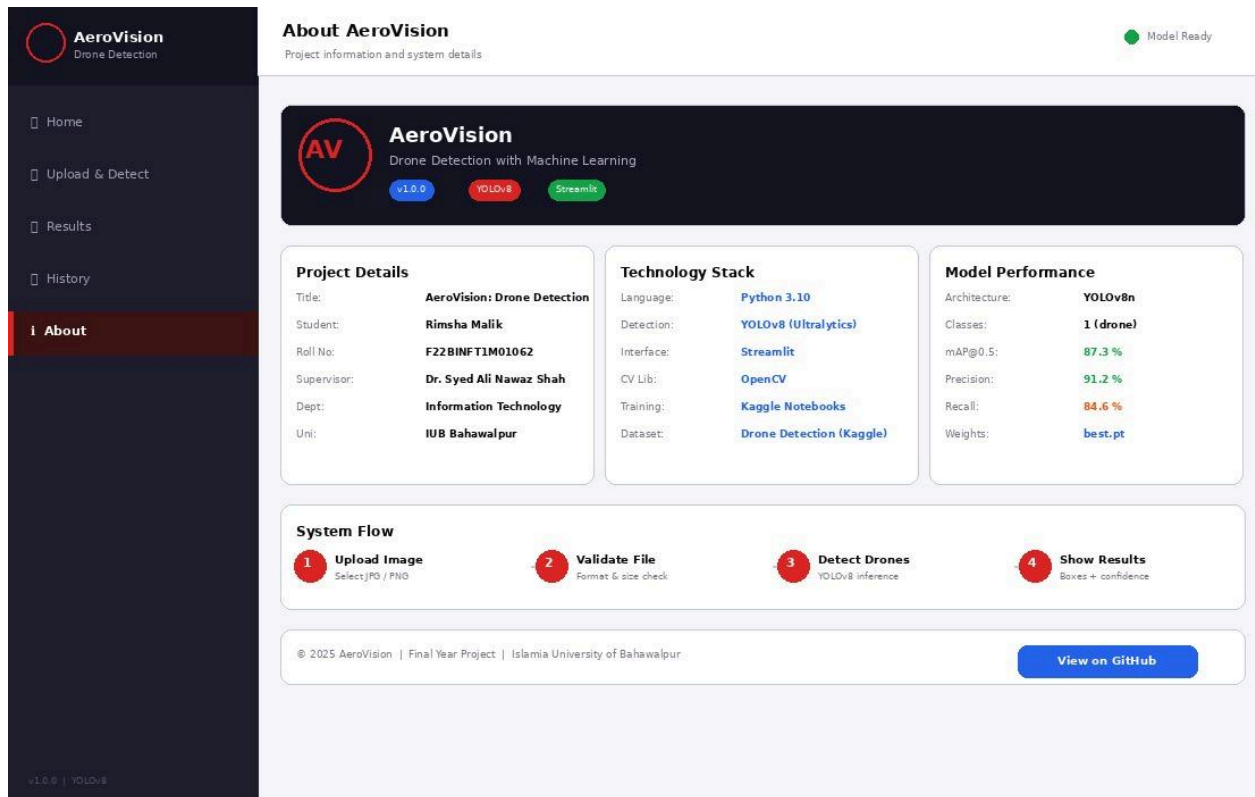


Figure 11 : Home Screen UI

Appendix A: Glossary

- **AeroVision:** The drone detection system developed as this Final Year Project
- **YOLOv8:** You Only Look Once version 8, the object detection model by Ultralytics used for drone detection
- **best.pt:** The saved YOLOv8 weight file produced after training on the benchmark dataset
- **mAP:** Mean Average Precision, the standard evaluation metric for object detection model accuracy
- **bbbox:** Bounding Box, the rectangle drawn around a detected drone in the annotated output image
- **session_state:** Streamlit mechanism for sharing Python variables across page navigations within a browser session
- **Inference:** The process of running a trained ML model on new input data to produce predictions
- **Confidence Score:** A float between 0 and 1 representing how certain the model is that a detected object is a drone
- **Annotation:** The process of drawing bounding boxes and labels on an image to mark detected objects
- **PIL:** Python Imaging Library (Pillow), used for opening and converting image files

Appendix B: Design Models

All design diagrams included in this SDD document:

- **Figure 1:** System Context Diagram (Section 1.6)
- **Figure 2:** Data Flow Diagram Level 0 and Level 1 (Section 1.6)
- **Figure 3:** Sequence Diagram — Drone Detection Flow (Section 1.6)
- **Figure 4:** Activity Diagram — Full System Control Flow (Section 1.6)
- **Figure 5:** Component Diagram — AeroVision Module Structure (Section 1.6)
- **Figure 6:** Class Diagram — AeroVision Classes and Relationships (Section 1.6)
- **Figure 6:** System Use Case Diagram — Interaction b/w User & Aerovision (Section 1.6)
- **Figure 7-11 :** UI Screen Mockups for all five pages (Section 3.4)

References

- [1] *G. Jocher et al., Ultralytics YOLOv8, GitHub repository, 2023. Available: <https://github.com/ultralytics/ultralytics>*
- [2] *IEEE Std 1016-2009, IEEE Standard for Information Technology Systems Design Software Design Descriptions, IEEE Computer Society, 2009*
- [3] *Streamlit Inc., Streamlit Documentation, 2023. Available: <https://docs.streamlit.io>*
- [4] *G. Bradski, The OpenCV Library, Dr. Dobbs Journal of Software Tools, 2000*
- [5] *Drone Detection Dataset, Kaggle. Available: <https://www.kaggle.com/datasets>*
- [6] *IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE Computer Society, 1998*